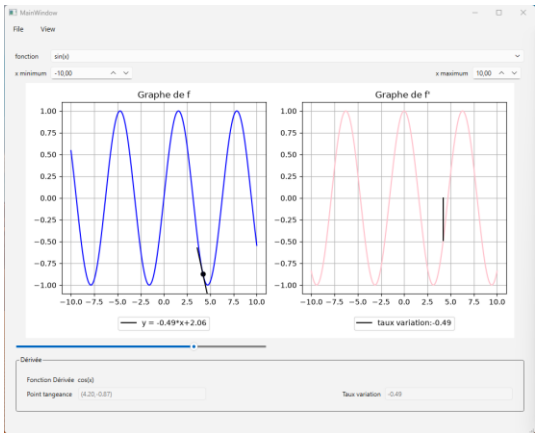
Cégep Limoilou

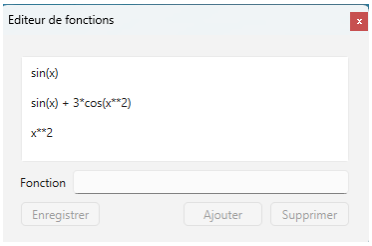
TP 2

420-SF3-RE

Objectif : Réaliser une application permettant d'afficher le graphe d'une fonction et de sa dérivée, ainsi qu'une interprétation de sa dérivée et de son taux de variation entre 2 bornes choisies par l'utilisateur.
On veut également pouvoir éditer la liste des fonctions qui peuvent être affichées et la sauvegarder dans un fichier json.

Voici une image indicative de cette application.





Consignes :

- Ce travail peut se réaliser seul ou à deux. Dans tous les cas son évolution doit être consignée dans un git, auquel l'enseignant doit pouvoir accéder. L'historique local doit donc être accessible avec le projet remis (dossier.git).
- Dans le cas d'une production à deux, la réalisation doit être partagée de manière relativement égale entre les deux participants. Au cas où ce ne serait pas le cas, la note individuelle reflétera cette différence. L'enseignant consultera et se basera sur votre historique de commit.
- Toute utilisation d'une IA doit être ciblée et ponctuelle, faire l'objet d'une citation claire, et être accompagnée d'une compréhension détaillée de chaque instruction, démontrée par des commentaires pertinents et explicatifs.

Remise :

- Le projet complet sur LEA, avec son dossier .git mais sans son venv trop volumineux. Vous devez conserver un historique de commit précis et soigné, qui sera utilisé au besoin le volume et la qualité du travail effectué par chacun.

Date : *dimanche 1 novembre à minuit*

Architecture et organisation du projet:

Votre programme doit contenir au moins 5 classes différentes :

- La fenêtre principale (Vue principale)
- La fenêtre de la liste des fonctions (Vue secondaire)
- Le Canvas Matplotlib pour afficher les graphes et les sommes (Vue secondaire)
- La classe qui maintient la liste des fonctions (Modèle)
- La classe qui maintient la fonction à intégrer et les propriétés associées (bornes, point de tangeance, etc.) (Modèle)s

Il est important de **respecter les responsabilités de chacune.** En particulier on vous demande autant que possible de séparer les vues des modèles. Aucun traitement lié à l'affichage ne doit se trouver dans les modèles, alors que la logique et les mathématiques doivent se trouver uniquement dans les modèles.

Étapes suggérées :

1) Créer la fenêtre principale avec un champ provisoire (`QLineEdit`) pour la fonction et un pour chaque borne (adapter l'exercice formatif)

2) Créer le canvas MPL, et implémenter le dessin du graphe dans le subplot de gauches. Lorsqu'on modifie la fonction ou une des bornes, la fonction doit se redessiner en tenant compte des nouvelles valeurs.

3) Ajouter le slider qui permet de choisir la première coordonnée du point de tangeance, le stocker dans le modèle et l'afficher dans le canvas MPL (subplot de fonction)

4) Ajouter le group box qui gère la dérivée, implémenter **la méthode du modèle** qui calcule le taux de variation et le point de tangence et afficher la première coordonnée du point de tangence.

5) Implémenter le dessin de la fonction dérivée dans le subplot de droite
6) Implémenter la représentation graphique de la tangeante et du taux de variation dans les deux subplot, les informations proposées dans l'image ci-dessus doivent figurer, mais la vous pouvez choisir votre propre manièsre de le faire
7) Ajouter une vue qui va permettre d'éditer une liste de fonctions (utiliser une `QListView`) avec un bouton ajouter et un bouton supprimer. Porter une attention particulière à la séparation vue modèle.
8) Implémenter la sauvegarde de la liste en json, puis le chargement de la liste depuis le json au démarrage de l'application.
9) Dans la vue principale remplacer le champ d'édition de la fonction par un ComboBox qui affiche au démarrage la liste des fonctions chargées dans le modèle à partir du json.

10) Implémenter ensuite les fonctionnalités résiduelles :
    - Mécanisme pertinent pour évider que xMin soit après xMax
    - Validation complète de la fonction à l'ajout (avec coloration du champ pour les expressions invalides)
    - Redimensionnement de fenêtres cohérent
    - Export de l'image en pdf (menu)
    - Affichage de la grille (menu checkable)
    - Gestion des couleurs (f, f' et t) (menu)
    - Possibilité de Docker la fenêtre d'édition de la liste de fonctions, de la montrer ou de la masquer (QDockWindow + menu)
    - Appliquer un css de votre choix
    - Toute autre bonne idée est bienvenue!

11) Effectuez des recherches pour afficher les fonctions de manière plus élégante, en utilisant LaTeX. Je vous suggère ici de vous faire aider par un outil IA de votre choix ou des recherches de votre cru, qui vont vous suggérer des solutions que vous devrez vous approprier, comprendre et commenter. Il est fondamental que vous démontriez ne pas avoir perdu la maitrise si vous utilisez un outil IA.

Remarque : l'affichage d'une fonction constante nécessite un peu de recherche, ne perdez pas trop de temps là-dessus, en effet si f est constante `f ([1,2,3])` retourne une seule valeur au lieu d'une liste de 3 valeurs. Pour résoudre ce problème, il suffit de remplacer :

`y = f(x)` par `y = np.broadcasts_to(f(x), x.shape)`